

Robust Drag-Aware Launch Planning and Shoot-on-the-Move Compensation for a Competition Mobile Robot

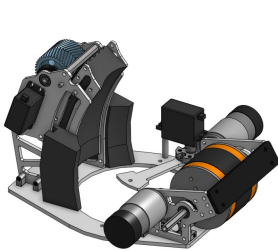
Eric Stoicescu

Quantum Robotics #14270, Bucharest, Romania
eric.stoicescu08@gmail.com

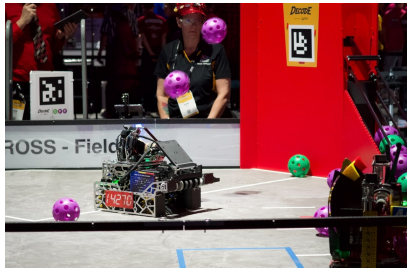
Abstract

A light perforated projectile is difficult to launch consistently from a moving robot because aerodynamic drag changes the flight, launcher commands vary, and the projectile inherits platform velocity. This paper develops and physically evaluates a unified pipeline comprising a quadratic-drag simulator, robust make-window selection, distance-indexed launcher maps, and shoot-on-the-move (SOTM) compensation. A vectorized fourth-order Runge–Kutta solver advances the command lattice; cubic Hermite interpolation and bracketed binary search locate the descending rim event. The deployed rule tests bounded perturbations around each speed–angle command and selects the nominal scoring command with the largest simulated make fraction. Caching the target-independent event map $X(v, \theta)$ and querying rectangular windows by prefix sums evaluates the same objective without repeated integration. The selected command changes by at most 0.125 m/s and 0.7° over four substantially different perturbation boxes and 19 target distances. Target entry increased from 70/100 to 93/100 in the development record (difference 23.0 percentage points, 95% CI 12.5–33.2). In four moving-speed strata, SOTM produced 54/60 entries versus 30/60 without compensation (difference 40.0 points, 95% CI 24.1–53.4; exact conditional stratified $p = 1.59 \times 10^{-7}$). Operational records contained 94/97 and 105/110 target entries. The resulting contribution is a reproducible model-to-deployment method with numerical, controlled, and competition-scale physical evidence.

Keywords: mobile robotics; projectile motion; aerodynamic drag; robust command selection; firing on the move; physical validation.



(a) Launcher CAD.



(b) Competition deployment.



(c) Tracked physical shot.

Figure 1: The launcher at the mechanical, deployment, and measurement levels. Images are from the team archive and the author’s tracking annotation.

1 Introduction

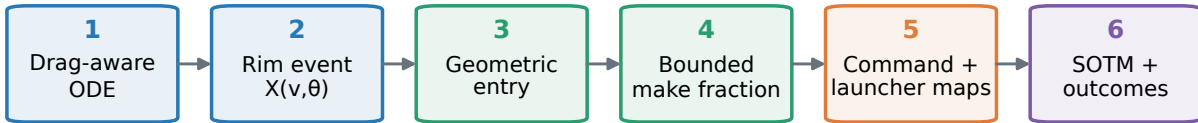
A stationary launch can be tuned by repetition. A moving-platform launch is harder because range, release speed, hood angle, platform motion, and launcher recovery all change the result. For the 2025–2026 FTC task considered here, vacuum ballistics provided a useful first estimate but did not explain the range-dependent error of a light perforated projectile. Selecting a single nominal trajectory was also fragile: a command near the front or rear lip could score in simulation and fail after a small release perturbation.

The system was therefore organized around the sequence in Figure 2: predict a drag-aware trajectory, reduce it to the descending rim event, apply one geometric score test, evaluate a bounded neighborhood of commands, and deploy the most tolerant command through the launcher maps. Platform motion is handled by shifting the aim point before the same distance-indexed maps are queried.

The paper makes three contributions:

1. an event-compressed drag simulator and robust selector that ranks commands by the made fraction of a bounded speed–angle neighborhood, with an exact event-map/prefix-sum implementation of that rule;
2. a deployed SOTM virtual-target controller with a contraction proof, tolerance corollary, acceleration-error bound, and a drag-based hypothesis for the measured gain $G = 0.9$;
3. validation that connects analytic and independent numerical references, offline robustness audits, controlled SOTM strata, and operational robot records while separating geometric entry from post-entry retention.

The integrated contribution is *drag-aware simulation + robust make-window selection + deployment + SOTM compensation + physical validation*. The season controller used the make-window principle in Section 5.5; the cached event map is its faster, decision-equivalent evaluation.



Cached event maps and prefix sums accelerate evaluation; they do not change the make-window objective.

Figure 2: The model-to-deployment workflow. Caching and prefix sums reduce evaluation cost while preserving the make-window objective.

2 Related work

Robotic throwing has been treated through sensitivity-aware trajectory design, model-based parameter optimization, and hybrid physics/data methods. Okada et al. [1] selected trajectories with reduced sensitivity to model uncertainty. TossingBot combined an analytic ballistic prior with a learned residual for previously unseen objects [2]. Luo et al. [3] studied multi-factor parameter optimization for a robotic projectile launcher. The present system differs in its small mobile-robot setting: commands are constrained by a measured launcher envelope, robustness is evaluated over a bounded speed–angle neighborhood, and the final system is connected to moving-platform compensation and competition data.

Dynamic throwing compensation has also been demonstrated with high-speed robot arms, where release state and target motion are coupled explicitly [4]. Here the actuator is a continuously aimed flywheel launcher on a translating base. The controller therefore shifts a virtual target through a measured time-of-flight map and must converge within the low-latency season control loop.

Quadratic drag for sports balls is well established [5, 6]. Perforations can produce orientation-dependent drag and lift, as shown by Rossmann and Rau [7]; this motivates using an effective coefficient and retaining physical calibration. Fourth-order Runge–Kutta, cubic Hermite interpolation, and adaptive ODE solvers are standard numerical tools [8, 9, 10]. Their role here is practical: they make a repeatable event calculation available to the selector.

Table 1: Principal notation.

Symbol	Meaning
v, θ	commanded projectile exit speed and launch angle
x, z	horizontal and vertical projectile position
$X(v, \theta)$	horizontal position of the descending rim-plane event
$J(v, \theta; d)$	simulated make fraction in a bounded perturbation box
t_d, t_f	release delay and rim-event time of flight
$\mathbf{p}_s, \mathbf{p}_g, \mathbf{p}_v$	shooter, real-target, and virtual-target positions
$\mathbf{v}_R, \mathbf{a}_R$	robot velocity and acceleration near release
G	inherited-velocity compensation gain
M, q	time-map Lipschitz bound and fixed-point contraction factor
η	projectile-to-flywheel surface-speed transfer coefficient

3 Physical system and outcome definitions

The projectile is a hollow perforated polypropylene sphere with nominal diameter $D = 0.127$ m and mass $m = 0.0748$ kg. Measured diameters were approximately 0.124–0.127 m. The launcher uses two counter-rotating nominal 6000 r/min motors with 1:1 gearing, 0.1384 m flywheels, and a servo-controlled hood. The measured command envelope used by the selector is 4.70–6.61 m/s and 42° – 58° . Launch height in the reported simulations is 0.400 m. The target’s top reference plane is 0.984 m above the floor and the opening depth is 0.465 m, following the season field geometry [12].

Two outcomes are kept separate throughout the paper. A *target entry* occurs when the projectile reaches the opening; a later bounce-out remains an entry. A *retained score* requires the projectile to enter and remain scored. This distinction prevents unmodeled spin and contact dynamics from being reported as errors of the flight model.

Robot pose and translational velocity are estimated from dead-wheel odometry and an inertial sensor. Those states index the target distance and SOTM correction, so the operational records evaluate localization, aiming, launcher response, and flight as one deployed chain. This paper makes no stand-alone localization accuracy claim; any pose error is therefore carried into the physical outcome.

4 Aerodynamic model

The state in the vertical launch plane is $\mathbf{y} = [x, z, v_x, v_z]^\top$. For horizontal wind v_w , air-relative velocity is $\mathbf{u} = (v_x - v_w, v_z)$ and $u = \|\mathbf{u}\|$. The equations used in the simulator are

$$\dot{x} = v_x, \quad \dot{z} = v_z, \quad (1)$$

$$\dot{v}_x = -ku(v_x - v_w), \quad \dot{v}_z = -g - kuv_z, \quad (2)$$

$$k = \frac{\rho C_D A}{2m}, \quad A = \pi(D/2)^2. \quad (3)$$

The implementation uses $g = 9.80665$ m/s², $\rho = 1.204$ kg/m³, and a constant effective $C_D = 0.47$. The density is the reference value for air near 20 °C and sea-level pressure; site atmospheric conditions were not logged. Reynolds number is reported diagnostically, but the deployed coefficient remains fixed throughout the integration.

Figure 3 places the drag effect inside the measured command envelope. At the representative 5.25 m/s, 58° command, the descending crossing moves from 2.084 m in vacuum to 1.859 m at $C_D = 0.47$, a 10.8% reduction. Changing C_D from 0.40 to 0.55 moves that event by 0.063 m. Drag therefore matters, while the stated coefficient sensitivity is smaller than the 0.338 m center-entry interval. Recomputing frontal area over the measured $D = 0.124$ – 0.127 m range moves the same crossing by 0.009 m. Robot-specific launcher maps provide the final system calibration; the simulator does not include spin, lateral forces, or goal-contact dynamics.

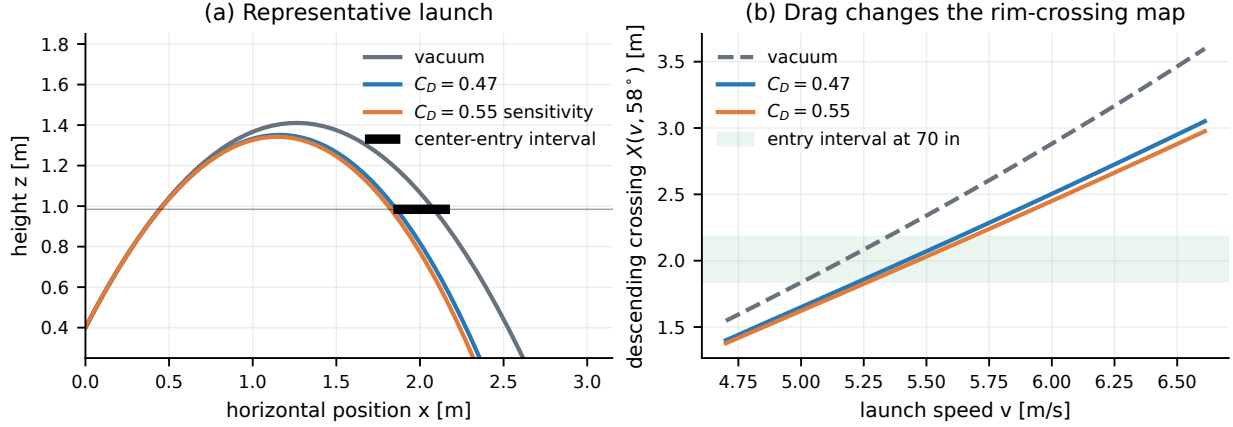


Figure 3: Effect of drag within the launcher envelope. The black segment in panel (a) is the center-entry interval, and panel (b) shows how drag changes the reusable rim-event map.

5 Trajectory simulator and robust launch selection

5.1 Why the simulator was developed

The simulator bridges a gap between a trajectory that can score once and a command that remains useful on hardware. Exit speed changes with flywheel recovery, battery state, projectile compression, and surface condition; hood and release geometry perturb the launch angle. A nominal shot close to either lip is therefore a poor calibration point even if its displayed path enters. The same program computes the trajectory, finds the scoring event, evaluates nearby commands, and exports a recommendation, avoiding inconsistent physics between those stages.

5.2 How one trajectory is calculated

For command (v, θ) and launch height z_0 ,

$$\mathbf{y}_0 = [0, z_0, v \cos \theta, v \sin \theta]^\top. \quad (4)$$

Equation (2) is advanced with classical fourth-order Runge–Kutta (RK4). The event lattice uses a fixed step $h = 0.02$ s; displayed paths use 0.005 s only to draw smoother curves.

Scoring depends on the instant when the projectile descends through the target plane, not on the nearest stored RK4 sample. Position and velocity at the two ends of the bracketing step define the cubic Hermite interpolant

$$\begin{aligned} H(\tau) = & (2\tau^3 - 3\tau^2 + 1)p_0 + (\tau^3 - 2\tau^2 + \tau)hd_0 \\ & + (-2\tau^3 + 3\tau^2)p_1 + (\tau^3 - \tau^2)hd_1, \quad 0 \leq \tau \leq 1. \end{aligned} \quad (5)$$

The step already provides opposite signs of $H_z(\tau) - z_{\text{rim}}$. Sixteen binary-search halvings retain the sign-changing half-interval, giving normalized width 2^{-16} . The bracketed solve cannot divide by a small derivative or leave the known interval. The same τ^* is used in H_x for crossing position and in the analytic derivative of Equation (5) for entry velocity.

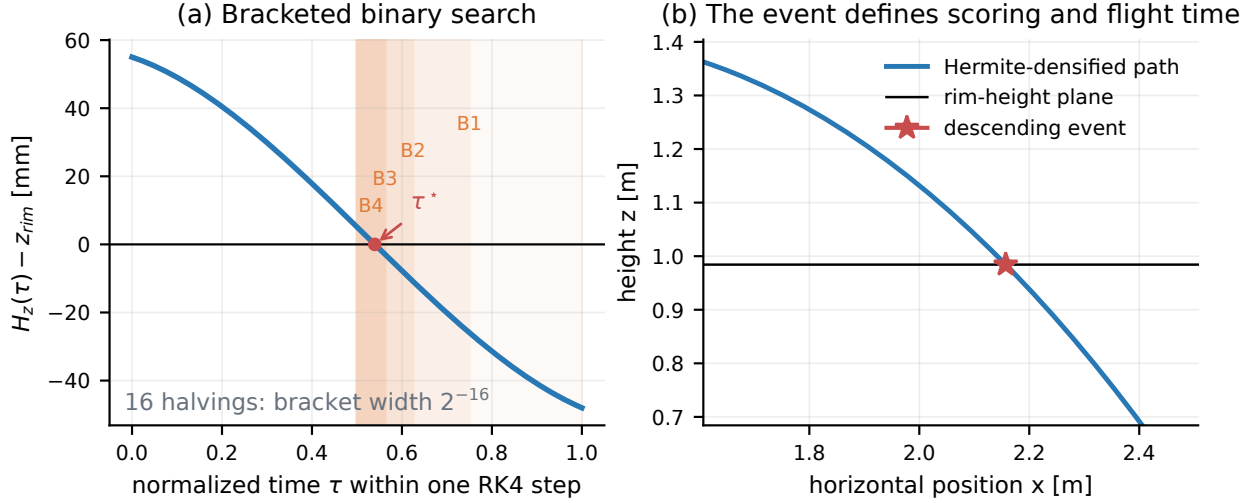


Figure 4: Hermite event reconstruction and bracketed binary search. The left panel shows the first four of 16 interval halvings; the right panel shows the descending event used for scoring and rim-event flight time.

5.3 Scoring criterion

The score test is intentionally geometric. Let d be the horizontal coordinate of the front lip, D_g the opening depth, and $r = D/2$. The projectile scores in the simulator if its center crosses the rim plane while descending and

$$d + r \leq X(v, \theta) \leq d + D_g - r. \quad (6)$$

For the reported geometry, Equation (6) gives a 0.338 m interval for the projectile center. Each command receives a binary geometric result, without an added probabilistic entry model. The simulator's scope ends at entry; subsequent bounce-out belongs to the retention outcome.

5.4 Calculating many trajectories efficiently

Two properties remove repeated work. First, NumPy arrays store one state per speed-angle pair, so every RK4 stage advances all active trajectories in one vectorized operation. A trajectory is removed after its descending rim event or when it can no longer score. Full paths are stored only for visualization.

Second, for fixed launch height, rim height, wind, and physical parameters,

$$X(v, \theta) = \text{horizontal position of the descending rim-plane event} \quad (7)$$

does not depend on target distance. Target placement changes only the interval in Equation (6). The current implementation therefore caches an exact event-map tile on the expanded lattice 4.00–7.30 m/s by 40.5°–59.5°, with steps of 0.025 m/s and 0.10°. The expanded limits contain every perturbation around a command in the measured 4.70–6.61 m/s by 42°–58° envelope.

Thresholding the cached X values produces a Boolean made/miss array for any target distance. A two-dimensional prefix sum

$$S(i, j) = \sum_{p \leq i} \sum_{q \leq j} I_{pq} \quad (8)$$

then returns the number of entries in any rectangular perturbation box using four array reads. Thus every candidate is evaluated exactly on the same lattice without reintegrating its neighbors. The map cache is keyed by every physical quantity that changes a trajectory and is protected for concurrent requests.

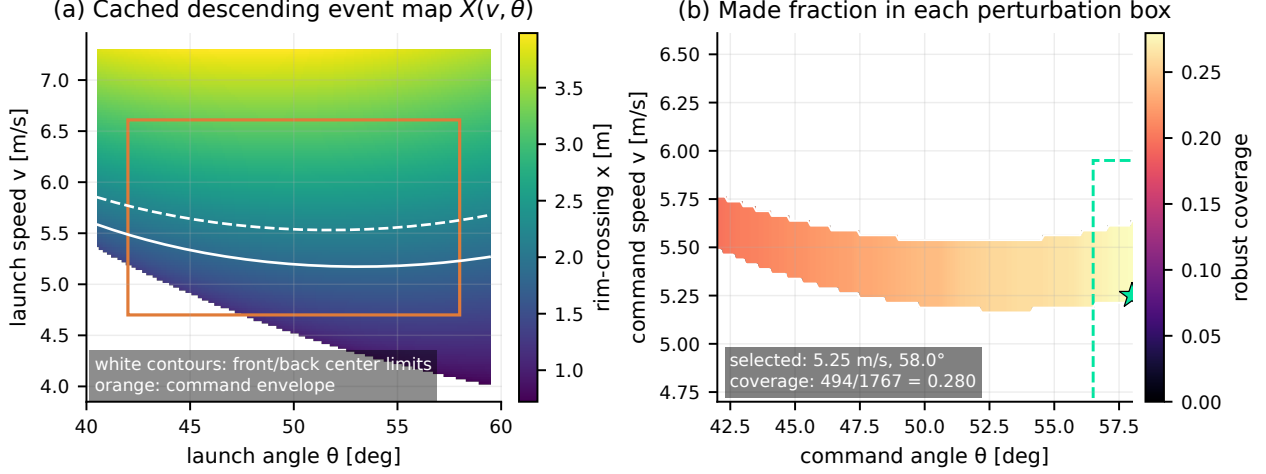


Figure 5: Cached event map and robust selection at 1.778 m (70 in). The star is the selected command. Its bounded box contains 494 scoring lattice points out of 1767, so its robust coverage is 0.280. This is a deterministic area fraction, used to rank commands.

5.5 Picking the best trajectory

For candidate (v, θ) and distance d , define

$$J(v, \theta; d) = \frac{1}{N} \sum_{i=1}^N I(v + \Delta v_i, \theta + \Delta \theta_i; d), \quad (9)$$

where $I = 1$ when Equation (6) is satisfied. The perturbations fill the bounded box $|\Delta v| \leq 0.7$ m/s and $|\Delta \theta| \leq 1.5^\circ$ on the lattice above. The selector considers only nominal commands that score, chooses the largest J , and breaks exact ties by lower speed and then lower angle. This is the deployed season rule in plain terms: test nearby speed–angle perturbations and choose the command with the highest simulated make fraction.

The two half-widths are engineering stress limits, not fitted standard deviations. The seven-shot launcher identification found a transfer coefficient of 0.259 ± 0.031 (sample standard deviation) across different speeds, angles, and distances. That 12% spread measures the error of one constant conversion across configurations; it is not shot-to-shot repeatability at a fixed command. Over the observed 4.70–6.61 m/s range it corresponds to 0.56–0.79 m/s, so 0.7 m/s was retained as a conservative map/model stress allowance. Repeated video annotation differed by about 0.3° ; the 1.5° half-width adds allowance for hood, release, and tracking variation that annotation alone does not measure.

The distinction explains the apparent mismatch between a 0.280 simulated coverage and the 93/100 physical entry result: J ranks commands over a wide, uniformly weighted stress domain and is not the predicted make probability. As an order-of-magnitude check only, a centered normal crossing error giving 93% inside the 0.169 m half-aperture would have standard deviation about 0.093 m. The local model sensitivity at the representative command is $\partial X / \partial v \approx 0.84$ m per (m/s), corresponding to about 0.11 m/s of fixed-command speed scatter. This is much narrower than the ± 0.7 m/s ranking box, as expected for a conservative stress test.

Dependence on the chosen box was audited offline rather than inferred from the coverage value. The complete selection was repeated at 19 distances from 60–105 in for half-width pairs $(0.3 \text{ m/s}, 0.5^\circ)$, $(0.5, 1.0^\circ)$, the deployed $(0.7, 1.5^\circ)$, and $(0.9, 2.0^\circ)$. Relative to the deployed box, the selected speed changed by at most 0.125 m/s and the angle by at most 0.7° ; every result remained inside the measured command envelope. Figure 11(a) shows that the distance–command trend is not created by one finely tuned box.



Figure 6: Simulator side view for the selected 1.778 m command. The interface reports the rim-event flight time, entry angle, score state, and make-window diagnostics.

The implemented sequence is:

1. integrate the expanded command lattice once and cache $X(v, \theta)$;
2. threshold the map with the requested target interval;
3. build the made/miss prefix sum;
4. evaluate every bounded box and select the nominal scoring maximum;
5. convert its ideal speed and angle through the calibrated launcher maps.

Representative simulator states are shown in Figure 7. The side view, tolerance map, and shot family all use the same 1.778 m target and selected 5.25 m/s, 58° command.

5.6 Numerical verification

Verification was separated from physical validation. With $C_D = 0$, 993 speed–angle cases were compared with the analytic parabola; maximum errors were $7.11 \times 10^{-7} \text{ m}$ in rim-crossing position and $1.52 \times 10^{-7} \text{ s}$ in event time. With drag enabled, 158 events were compared with an independently implemented SciPy DOP853 solve using strict tolerances, event detection, and a 2 ms maximum step. Maximum errors were $5.41 \times 10^{-7} \text{ m}$ and $1.55 \times 10^{-7} \text{ s}$. The cached event tile matched direct batched integration, and an exhaustive search at six representative distances returned exactly the same command and make count as the prefix-sum selector. These tests establish correct implementation of Equation (2); physical identification of C_D is a separate calibration question.

On the test machine, building a new exact event map required approximately 68 ms; warm selection runs were 7–9 ms. Timing is reported only to show that the complete lattice can be used interactively. Figure 8 also confirms that selected commands remain inside the measured launcher envelope.

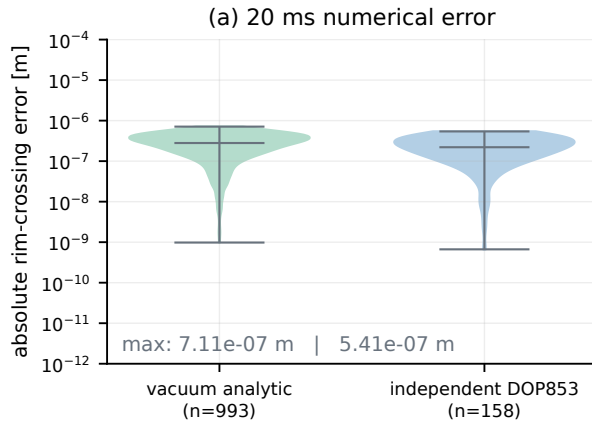


(a) Bounded tolerance map.

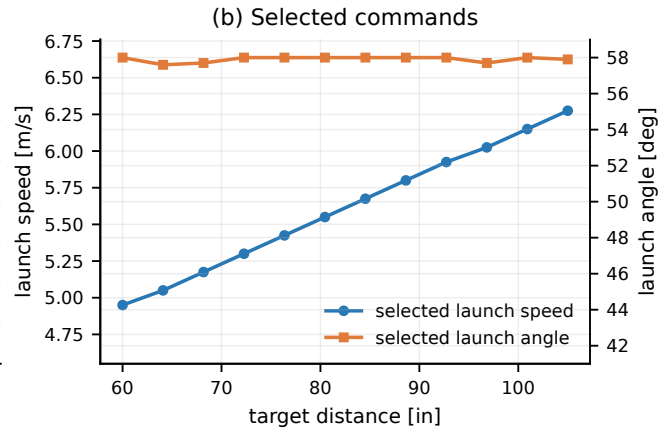


(b) Family of nominal scoring shots.

Figure 7: Representative simulator diagnostics. The screenshots document the actual interface used to inspect the same scoring rule; quantitative values are reported separately in Figure 5.



(a) 20 ms numerical error



(b) Selected commands

Figure 8: Independent numerical checks and selected commands over 1.524–2.667 m (60–105 in). Error distributions are plotted on a logarithmic scale.

5.7 Lookup-table deployment

The ideal command selected by the simulator must be converted to actuator coordinates. The season controller therefore uses distance-indexed maps for flywheel target speed, hood position, and rim-event time of flight. Knots were set from simulator guidance and physical calibration. Between knots, a shape-preserving cubic Hermite interpolant is used so a locally flat calibration segment cannot develop the overshoot of an unconstrained cubic spline. This is the standard monotone construction of Fritsch and Carlson [11]; it serves as the interpolation layer for deployment.

The robot first estimates the target distance, queries all three maps, waits for the flywheel and hood to enter their ready bands, and releases. The ideal exit speed is not sent directly to a motor: physical calibration absorbs wheel slip, compression, surface condition, and motor response. Consequently, the simulator defines the command geometry while the maps define the robot-specific actuator conversion.

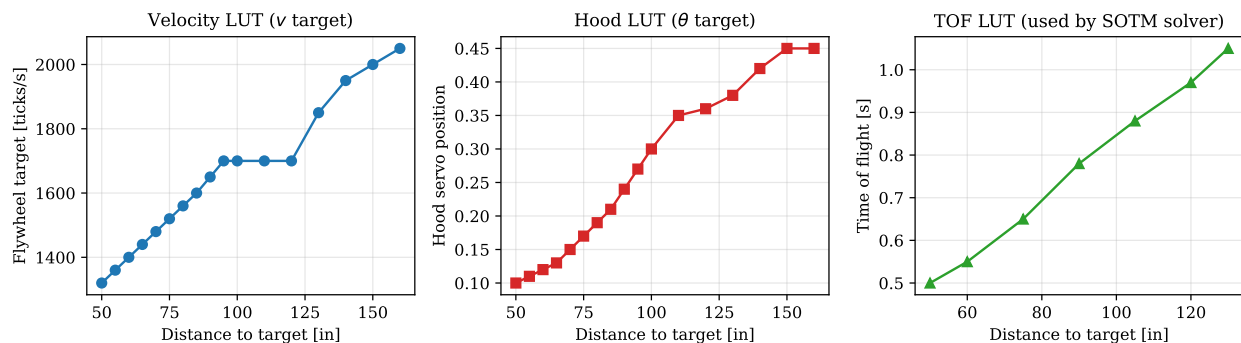


Figure 9: Distance-indexed season launcher maps. The time-of-flight map stores the descending rim-event time, not the later end of the displayed path.

6 Shoot-on-the-move compensation

The implementation described in this section is frozen by the public pre-MTI robot snapshot [75db349d](#), dated 17 June 2026 and labeled “final values before MTI.” Its `Sensors.java` records $G = 0.9$, $t_d = 0.05$ s, $\epsilon = 0.05$ s, five updates, the seven time-of-flight knots used below, and zero acceleration feed-forward. This snapshot, rather than a later branch, ties the reported season results to a single implementation state.

6.1 Virtual target

When the robot moves, the projectile inherits part of the platform velocity. Let $\mathbf{p}_s$ be shooter position, $\mathbf{p}_g$ the real target, $\mathbf{v}_R$ platform velocity, t_d release delay, and t_f rim-event flight time. The controller aims at

$$\mathbf{p}_v = \mathbf{p}_g - G\mathbf{v}_R(t_d + t_f), \quad (10)$$

where $G = 0.9$ is the deployed inheritance gain. The sign shifts the virtual target opposite the platform motion. Stationary operation is an immediate sanity check: as $\|\mathbf{v}_R\| \rightarrow 0$, the correction vanishes.

6.2 Time-of-flight iteration

Time of flight depends on the virtual distance, which itself depends on time of flight. With $d(\mathbf{p}) = \|\mathbf{p} - \mathbf{p}_s\|$, the scalar update is

$$t_{k+1} = F(t_k) = t_f \left(\left\| \mathbf{p}_g - G\mathbf{v}_R(t_d + t_k) - \mathbf{p}_s \right\| \right). \quad (11)$$

The table is clamped to its calibrated distance interval. The deployed solver starts at the real-target time, stops when consecutive times differ by at most $\epsilon = 0.05$ s, and caps the calculation at five updates.

6.3 Contraction proof, tolerance corollary, and omitted dynamics

Let the clamped time map be Lipschitz with $M = \sup_d |t'_f(d)|$. For any t and s , the reverse triangle inequality gives

$$|F(t) - F(s)| \leq M \left| \left\| \mathbf{p}_g - G\mathbf{v}_R(t_d + t) - \mathbf{p}_s \right\| - \left\| \mathbf{p}_g - G\mathbf{v}_R(t_d + s) - \mathbf{p}_s \right\| \right| \quad (12)$$

$$\leq G\|\mathbf{v}_R\|M|t - s| = q|t - s|. \quad (13)$$

Clamping makes the time interval invariant. Therefore, if $q = G\|\mathbf{v}_R\|M < 1$, Banach’s fixed-point theorem gives one fixed point and convergence from every initial time in the interval. The exact Hermite segments of the 0.50–1.05 s season table give $M = 0.4867$ s/m. With $G = 0.9$, the sufficient speed bound is

$$\|\mathbf{v}_R\| < \frac{1}{GM} = 2.28 \text{ m/s}. \quad (14)$$

All controlled tests through 1.7 m/s lie inside this bound; at 1.7 m/s, $q \leq 0.745$.

The proof also supplies an a-posteriori tolerance bound. If the returned iterate satisfies $|t_k - t_{k-1}| \leq \epsilon$, successive increments contract geometrically, hence

$$|t_k - t^*| \leq \frac{q\epsilon}{1 - q}, \quad \|\mathbf{p}_v^{(k)} - \mathbf{p}_v^*\| \leq G\|\mathbf{v}_R\| \frac{q\epsilon}{1 - q}. \quad (15)$$

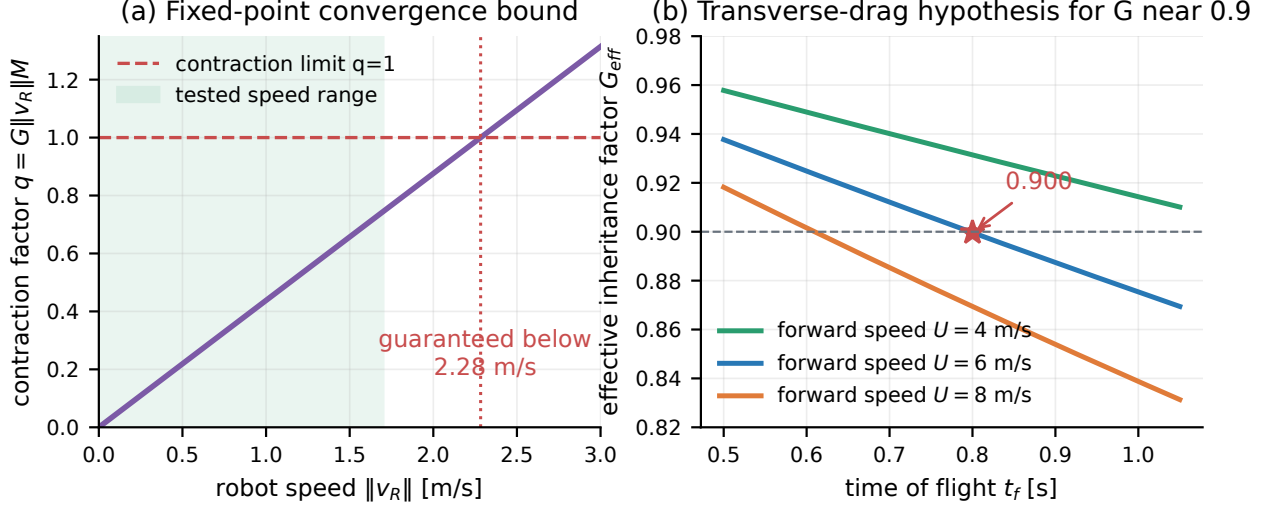


Figure 10: SOTM analysis. Panel (a) shows the sufficient contraction bound over the tested speed range. Panel (b) shows the transverse-drag hypothesis for an effective inheritance gain near 0.9.

If the iteration cap is reached first, the same formula applies with the final observed increment in place of ϵ . At 1.7 m/s and $\epsilon = 0.05$ s, the worst-case theorem gives 0.146 s and 0.223 m; this is a conservative guarantee, not the realized controller error. An exact offline replay of the archived Java update was therefore evaluated on 1,399,392 cases: 60–116 in at 0.5 in spacing, 0–1.7 m/s at 0.02 m/s spacing, and every relative motion direction at 2.5° spacing. Every case met the deployed stopping threshold by the fifth update; only 170 cases required all five. Relative to a fully converged solution of the same clamped table, the virtual-target position error was 0.083 m at the 99th grid percentile and 0.171 m in the single worst case, while the largest returned fixed-point residual was 0.031 s. These are deterministic grid summaries, not sampling probabilities. Above 2.28 m/s the theorem makes no convergence claim, so operation outside the certified range requires a residual guard or a bracketed scalar solve.

Equation (10) assumes constant velocity near release. If acceleration $\mathbf{a}_R$ persists over the delay, the neglected first-order displacement is bounded by

$$\|\delta \mathbf{p}_a\| \leq \|\mathbf{a}_R\| \left(t_d t_f + \frac{t_d^2}{2} \right). \quad (16)$$

For $t_d = 0.05$ s and $t_f = 0.50$ – 1.05 s, this equals 2.6–5.4 cm per 1 m/s^2 . The value explains why performance can decrease when acceleration changes rapidly even though velocity compensation remains active.

The fitted value $G = 0.9$ also has a physical interpretation. Linearizing drag on a small transverse inherited-velocity component w about projectile forward speed U gives $\dot{w} = -kUw$. Averaging the inherited displacement over the delay and flight yields

$$G_{\text{eff}} = \frac{t_d + (1 - e^{-kUt_f})/(kU)}{t_d + t_f}. \quad (17)$$

With $k = 0.0479 \text{ m}^{-1}$, $U = 6 \text{ m/s}$, and $t_f = 0.8 \text{ s}$, $G_{\text{eff}} = 0.900$. Over $U = 4$ – 8 m/s and $t_f = 0.50$ – 1.05 s , Equation (17) spans 0.831–0.958. This is an algebraic hypothesis for why a gain near 0.9 works. Fitted inheritance identification and a fully drag-coupled inertial launch model remain distinct modeling tasks.

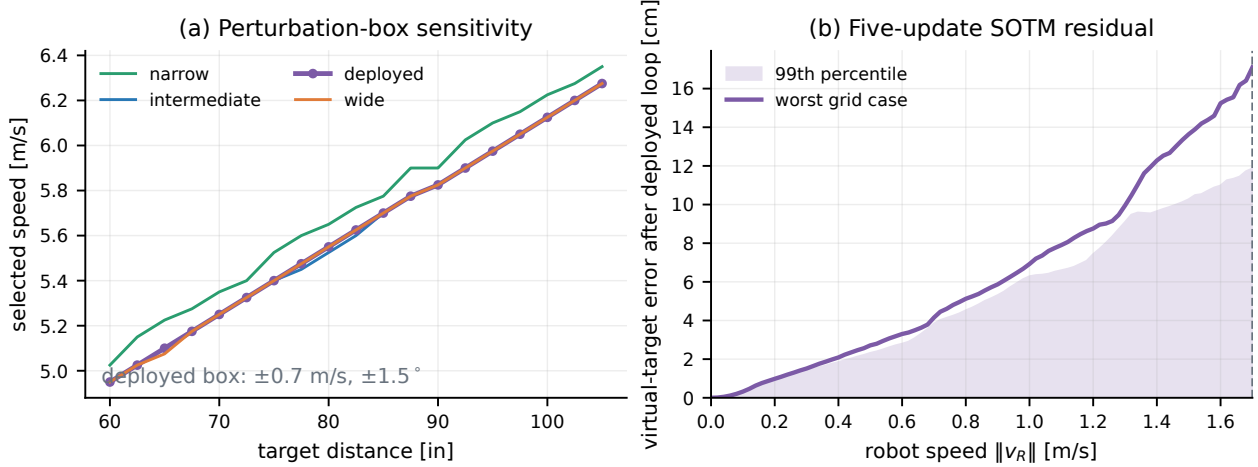


Figure 11: Offline audits of the two bounded design choices. Panel (a) repeats the selector for four perturbation boxes; panel (b) compares the archived five-update SOTM output with the converged fixed point over the complete test distance and speed domain. The shaded quantity is a deterministic grid percentile.

7 Experimental methods

7.1 Datasets and outcomes

Four records evaluate the deployed pipeline. The developmental record contains 100 initial-calibration launches followed by 100 launches after make-window calibration. The solo operational record contains 97 launches. A complete, continuously filmed MTI match contains 110 launches and provides event-level evidence of the final system under competition conditions. The selection criterion was completeness and continuity of the public recording. Finally, the SOTM ON/OFF record contains 15 launches per state at stationary, 0.5, 0.8, 1.5, and 1.7 m/s commanded platform speed.

Every physical launch is assigned one terminal outcome: retained score, entry followed by bounce-out, or geometric miss. The first two categories form the target-entry count. The development and operational results use the final season make-window workflow. The cached event-map implementation receives no separate empirical attribution because it only accelerates the same selection rule.

7.2 SOTM protocol and aggregate record

The controlled comparison focused on deceleration, the normal season release state. ON and OFF conditions were alternated within each commanded-speed block, with 15 launches per state. The test design traversed 60–116 in in 4 in steps, once per distance in each cell. The preserved outcome record contains the cell totals, not the mapping of outcomes to distance, shot order, timestamps, or measured release-state logs. Consequently, inference is conditioned on the five commanded-speed blocks; no post hoc telemetry covariates are introduced.

7.3 Statistical analysis

Binomial proportions are reported with two-sided Wilson 95% intervals. Differences between independent proportions use Newcombe’s Wilson-score method. For the four moving SOTM strata, a continuity-corrected Cochran–Mantel–Haenszel (CMH) test conditions on commanded speed; the common odds ratio is the Mantel–Haenszel estimate. The stationary stratum is excluded from the test because both conditions are 15/15 and therefore contains no treatment contrast. Because several

cells are close to complete separation, an exact conditional stratified test is also computed by convolving the four within-stratum hypergeometric distributions. The two-sided probability-ordering tail is reported as the primary p value. Both calculations use only the preserved cell counts.

7.4 Computational workflow and AI assistance

All numerical values were recomputed from the source equations, implementation, and reported count tables. OpenAI Codex was used during revision for source-code review, plotting-code generation, language editing, and consistency checks. It did not create experimental observations. The author reviewed the code, figures, and manuscript and remains responsible for their content.

8 Results

8.1 Development and operational deployment

The sequential development record improved from 70/100 target entries (70.0%, 95% CI 60.4–78.1) to 93/100 (93.0%, 86.3–96.6). The difference was 23.0 percentage points (Newcombe 95% CI 12.5–33.2). Because the stages were sequential rather than randomized, this result supports the complete make-window calibration workflow; component-level attribution lies outside the sequential design.

The solo operational record contained 94/97 entries (96.9%, 91.3–98.9) and 84/97 retained scores (86.6%, 78.4–92.0). Ten of its 13 non-retained outcomes were bounce-outs. The selected MTI match contained 105/110 entries (95.5%, 89.8–98.0) and 101/110 retained scores (91.8%, 85.2–95.6). This decomposition shows that residual post-entry contact was a material failure mode even when the geometric trajectory reached the opening.

8.2 Model-to-observation consistency

The simulator’s most direct physical role is ranking feasible commands before robot calibration. The measured 93% development-stage rate is consistent with a much narrower fixed-command error than the conservative stress box, as the scale check in Section 5.5 shows. Meanwhile, the representative $C_D = 0.40$ – 0.55 sensitivity moves the crossing by 0.063 m, smaller than both the 0.338 m aperture and the speed-stress effect (approximately -0.586 to $+0.601$ m for $\Delta v = \mp 0.7$ m/s at the representative command). This scale separation supports the deployed architecture: retain drag in the flight model, then absorb robot-specific systematic offsets in the calibrated launcher maps.

8.3 Controlled SOTM ON/OFF experiment

Stationary entry was identical at 15/15 with SOTM OFF and ON, as predicted by Equation (10). At moving speeds, OFF produced 30/60 entries (50.0%, 95% CI 37.7–62.3) and ON produced 54/60 (90.0%, 79.9–95.3). The difference was 40.0 percentage points (Newcombe 95% CI 24.1–53.4). Conditioning on the four commanded-speed strata gave continuity-corrected CMH $\chi^2 = 24.82$, $p = 6.3 \times 10^{-7}$, and common odds ratio 21.0. The exact conditional stratified test gave two-sided

Table 2: Operational outcome decomposition.

Dataset	Launches	Entries	Retained	Bounce-outs / misses
Solo operational aggregate	97	94	84	10 / 3
Selected MTI match	110	105	101	4 / 5

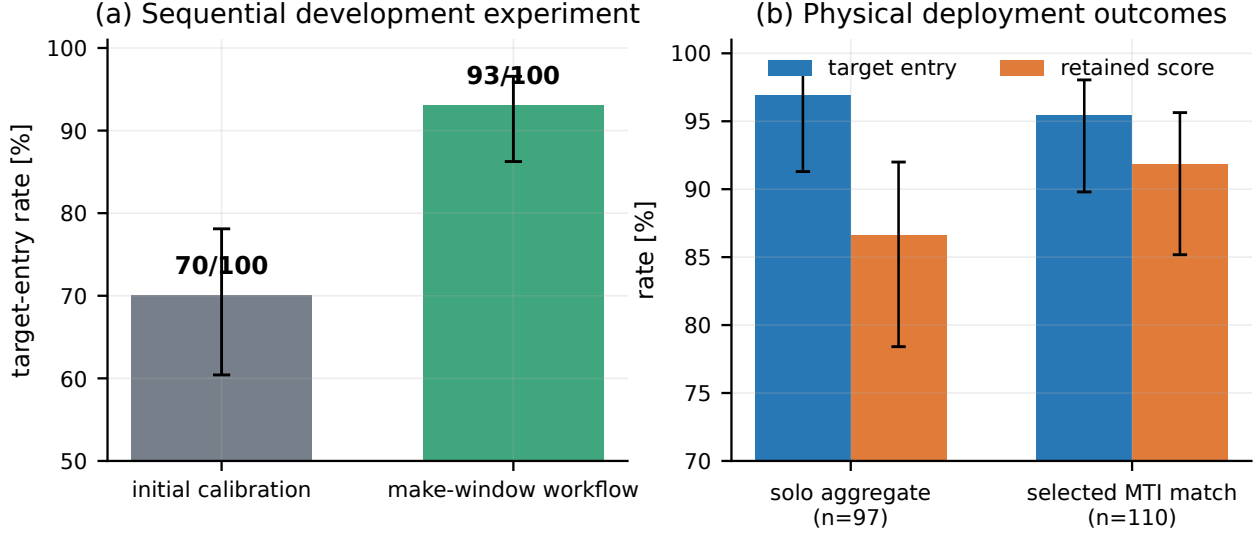


Figure 12: Physical outcomes with Wilson 95% intervals. Panel (a) is the sequential 100+100 development record. Panel (b) separates target entry from retained score in operational deployment.

Table 3: Target-entry counts in the SOTM experiment; 15 launches per cell.

State	Stationary	0.5 m/s	0.8 m/s	1.5 m/s	1.7 m/s
SOTM OFF	15/15	13/15	10/15	5/15	2/15
SOTM ON	15/15	15/15	15/15	13/15	11/15

$p = 1.59 \times 10^{-7}$ (directional upper-tail $p = 7.97 \times 10^{-8}$), in close agreement with the CMH conclusion.

The gap increases with speed: 13.3 percentage points at 0.5 m/s, 33.3 at 0.8 m/s, 53.3 at 1.5 m/s, and 60.0 at 1.7 m/s. SOTM itself declines from 15/15 at 0.8 m/s to 11/15 at 1.7 m/s. Equation (16) provides a quantitative candidate mechanism: rapid acceleration changes can add several centimeters of unmodeled lead error per m/s^2 , alongside latency and velocity-estimation error. Because the preserved outcome record is aggregate, the effect estimate is interpreted at the condition level; telemetry-conditioned modeling lies outside the available record.

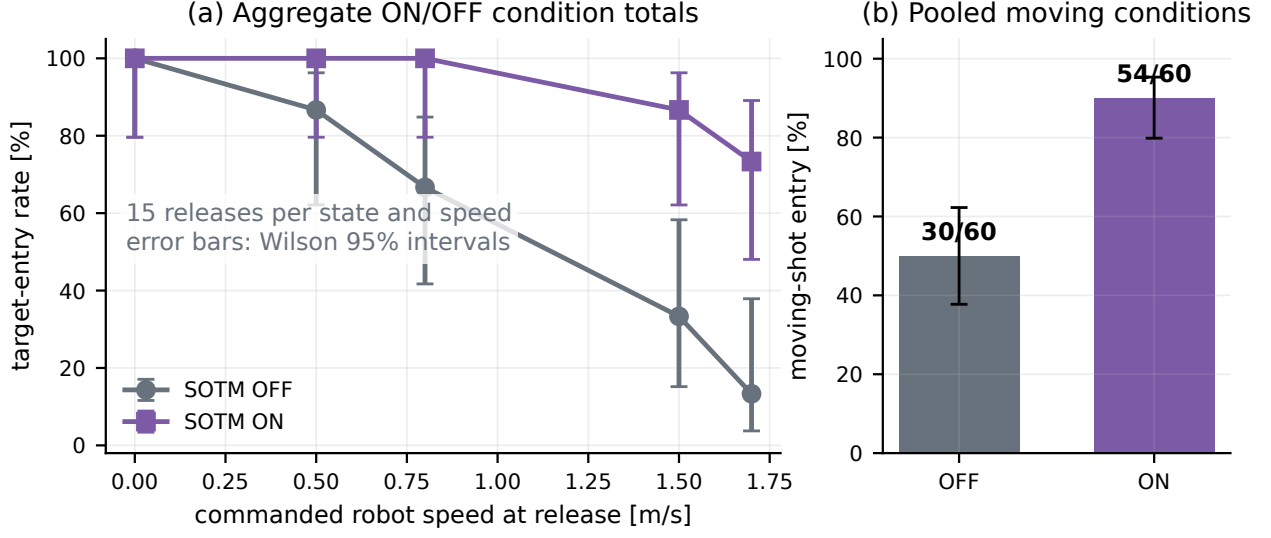


Figure 13: SOTM target-entry rate by commanded platform speed. Error bars are Wilson 95% intervals. The CMH analysis uses the four moving-speed strata.

9 Pilot launcher identification

Seven launches were recorded at 240 frames/s with a 1.000 m reference scale. Projectile centers were marked frame by frame, yielding fitted exit speed and launch angle. With flywheel radius $r_{\text{fw}} = 0.0692$ m and 28 encoder counts/revolution, the transfer coefficient was defined by

$$v_{\text{exit}} = \eta \left(2\pi r_{\text{fw}} \frac{\omega_{\text{ticks}}}{28} \right). \quad (18)$$

The mean was $\bar{\eta} = 0.259$, sample standard deviation 0.031, and 95% t interval 0.230–0.288. Because speed, angle, and distance changed across the seven launches, this variation is model/calibration scatter across configurations and is not used as a fixed-command repeatability estimate.

The tracking sequence is used for launcher-scale identification, not to infer C_D : the camera was not calibrated to the true flight plane. Keeping these roles separate prevents a two-dimensional image fit from being presented as an aerodynamic measurement.



Figure 14: One 240 frames/s tracking sequence. Markers identify manually selected projectile centers used for the pilot launcher calibration.

10 Discussion

The tests show how the components address different error sources. Drag changes the rim-crossing map enough to matter; robust make-window selection avoids commands that score only at one nominal point; the chosen commands can be deployed through calibrated launcher maps; and SOTM compensation preserves entry as platform speed rises. The cached event map strengthens the implementation by reusing the same physics across target distances, while prefix sums make the exact bounded search fast enough for interactive calibration.

The entry/retention split also changes the next design priority. In the solo record, 10 of 13 non-retained outcomes had already entered. Those events cannot be repaired solely by changing the geometric trajectory test; marked-projectile spin measurement and goal-contact analysis are the appropriate follow-up.

10.1 Scope of inference

The evidence supports three claims at deliberately different levels. The analytic and DOP853 comparisons validate the numerical event calculation. The 70/100 to 93/100 development record validates the integrated calibration workflow, while its sequential design does not assign the improvement to one isolated software line. The SOTM cells support a speed-stratified condition effect under both CMH and exact conditional analysis; without shot-order or release-state logs, they are not used to fit a telemetry-level causal model. The complete filmed MTI match is deployment evidence rather than an estimator of a season-wide rate.

Likewise, the two-dimensional constant- C_D model predicts target entry, not spin-dependent goal contact. Pose-estimation error, actuator error, and any unmodeled lateral motion remain inside the end-to-end physical outcomes. The entry/retention split makes this boundary observable: it preserves a strong test of launch geometry without crediting the flight model for bounce-out dynamics it does not contain.

11 Reproducibility and data availability

The simulator source is publicly available in the `decode_sim/decode_sim` directory of <https://github.com/stoicescueric/Trajectory-Simulation>. The implementation analyzed for this manuscript is identified by the SHA-256 checksums below.

```
physics.py
841ccb409fed7db93d319aa4e054c3002abd538519a0b1c3e537e060893049c2
verify.py
ec2dfa7661c5e7c00b6b2a1ae81f87040cb1dece3173ff8122ee5747936f0fb7
```

The exact season SOTM controller is preserved independently at robot commit [75db349d](#). The versioned supplementary archive supplied with the submission contains the two numerical source files, both independent-validation scripts, the aggregate SOTM count table, its 60–116 in design schedule, the exact stratified analysis, the perturbation-box audit, the five-update SOTM replay, machine-readable results, and a SHA-256 manifest. No shot-level SOTM outcomes or telemetry are claimed or reconstructed.

The public MTI footage is available at <https://www.youtube.com/watch?v=KaQI9TnmK7E>, and supplementary autonomous footage at <https://www.youtube.com/watch?v=lQc21buvXC8>. The archive includes validated Zenodo metadata; its DOI is inserted in this paragraph immediately after the corresponding author’s authenticated deposit, before resubmission.

12 Conclusion

This work connects drag-aware trajectory calculation to robust command selection and moving-platform deployment on a physical robot. The final selector tests nearby (v, θ) perturbations and chooses the scoring command with the highest simulated make fraction. The cached $X(v, \theta)$ event map and prefix sum evaluate that same rule efficiently and preserve its selected command. Numerical checks against analytic vacuum motion and an independent adaptive ODE solver agree at sub-micrometer rim-event scale.

Physical results align with the intended roles of the two controls. The make-window workflow increased sequential development entry from 70/100 to 93/100. In the moving-speed SOTM strata, compensation increased entry from 30/60 to 54/60 (exact conditional stratified $p = 1.59 \times 10^{-7}$), while the stationary control remained 15/15 in both states. Perturbation-box sensitivity preserved the distance–command trend, and exact replay of the season SOTM loop met its stopping tolerance throughout the test domain. The contraction proof covers the tested speed range, the acceleration bound explains why aggressive deceleration can still degrade performance, and the transverse-drag calculation provides a testable explanation for $G \approx 0.9$. The resulting system is an integrated simulator, robust selector, deployment method, motion compensator, and physically validated launcher workflow.

Acknowledgments

The author thanks the members and mentors of Quantum Robotics for the shared robot platform, mechanical development, field operation, and season calibration. OpenAI Codex (accessed August 2026) assisted with code review, figure-generation support, and language editing; the author verified and edited all output.

Conflicts of Interest

The author declares no conflict of interest.

References

- [1] M. Okada, A. Pekarovskiy, and M. Buss, “Robust trajectory design for object throwing based on sensitivity for model uncertainties,” in *Proc. IEEE International Conference on Robotics and Automation*, pp. 3089–3094, 2015. doi: 10.1109/ICRA.2015.7139623.
- [2] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, “TossingBot: Learning to Throw Arbitrary Objects with Residual Physics,” *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307–1319, 2020. doi: 10.1109/TRO.2020.2988642.
- [3] J. Luo, Y. Chen, Y. Cheng, J. Lin, Z. Wang, and X. Yin, “Study on robotic projectile launching based on multi-factor analysis and parameter optimization,” *Frontiers in Mechanical Engineering*, vol. 11, art. 1707301, 2025. doi: 10.3389/fmech.2025.1707301.
- [4] A. Takahashi, M. Sato, and A. Namiki, “Dynamic compensation in throwing motion with high-speed robot hand-arm,” in *Proc. IEEE International Conference on Robotics and Automation*, pp. 6287–6292, 2021. doi: 10.1109/ICRA48506.2021.9560866.
- [5] E. Achenbach, “Experiments on the flow past spheres at very high Reynolds numbers,” *Journal of Fluid Mechanics*, vol. 54, no. 3, pp. 565–575, 1972. doi: 10.1017/S0022112072000874.
- [6] R. D. Mehta, “Aerodynamics of sports balls,” *Annual Review of Fluid Mechanics*, vol. 17, pp. 151–189, 1985. doi: 10.1146/annurev.fl.17.010185.001055.
- [7] J. Rossmann and A. Rau, “An experimental study of Wiffle ball aerodynamics,” *American Journal of Physics*, vol. 75, no. 12, pp. 1099–1105, 2007. doi: 10.1119/1.2787013.
- [8] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Chichester, UK: Wiley, 2016.
- [9] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, 2nd ed. Berlin, Germany: Springer, 1993.
- [10] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [11] F. N. Fritsch and R. E. Carlson, “Monotone piecewise cubic interpolation,” *SIAM Journal on Numerical Analysis*, vol. 17, no. 2, pp. 238–246, 1980. doi: 10.1137/0717021.
- [12] FIRST, *FIRST Tech Challenge DECODE Presented by RTX Competition Manual*, 2025–2026 season.